مدن المهن و الكفاءات
ⵜⵉⵎⴰⵣⵉⵖⵉⵏ ⵏ ⵜⵎⵙⵙⵓⵔⵉⵏ ⴰ ⵜⵓⵣⵔⴰⵡ
Cités des métiers et des compétences

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle et de la Promotion du Travail

# Module: Développer en back-end

## Manipulation des contrôleurs

**Filière: Développement digital – option web full stack**

## Qu’est-ce qu’un contrôleur ?

Dans le modèle **MVC** (modèle-vue-contrôleur), le contrôleur contient la logique concernant les actions effectuées par l’utilisateur.

En pratique, dans une application **Laravel**, l’utilisation de contrôleurs permet **de libérer les routes** du code qu’elles contiennent dans leurs fonctions de rappel.

Un **contrôleur** est matérialisé par une classe et chacune de ses méthodes représente une action. Une action correspond généralement à une route.

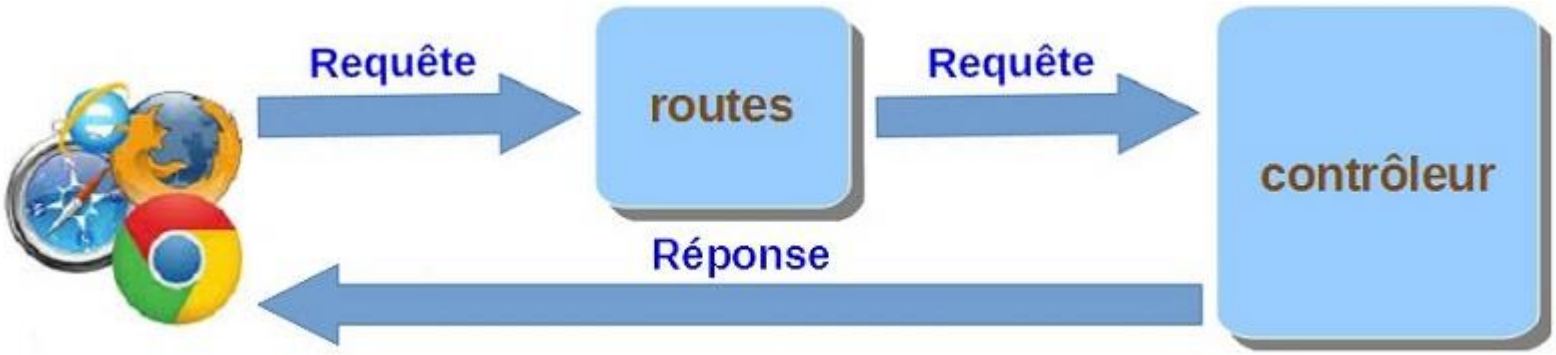
Les contrôleurs sont destinés à regrouper la logique de traitement des demandes associée dans une seule classe.

Dans votre projet Laravel, ils sont stockés dans le répertoire **app/Http/Controllers**.



## Rôle d'un contrôleur

La tâche d'un **contrôleur** est de recevoir une requête (qui a déjà été sélectionnée par une route) et de définir la réponse appropriée, rien de moins et rien de plus. Voici une illustration du processus :



## Créer un contrôleur sous Laravel

Pour créer un contrôleur à partir de la fenêtre du terminal, ouvrez le terminal et changez le répertoire vers votre dossier racine laravel. Une fois que vous y êtes, vous pouvez exécuter l'une des commandes suivantes pour créer un contrôleur.

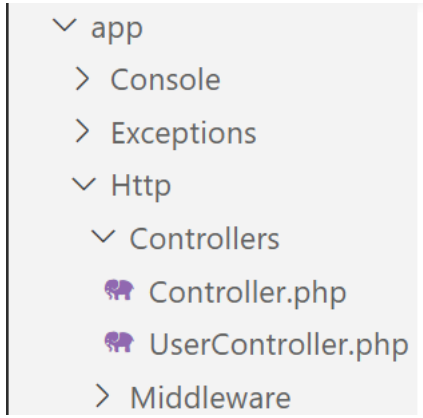
**Artisan**, l'outil en ligne de commande fourni avec Laravel, permet de créer rapidement un fichier contenant la structure de base d'un contrôleur.

### Créer un simple contrôleur vierge

- Pour créer un contrôleur vous pouvez directement entrer la commande : **php artisan make:controller NomDeMonController**

***Exemple***:

**php artisan make:controller UserController**



## Créer un contrôleur sous Laravel

Dans l'exemple précédent nous avons créer le controller **UserController**.

Le contrôleur crée est sous le dossier **app/Http/Controllers**

Ajoutons la méthode ***index*** au controlleur ***UserController*** :

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    //
    public function index() {
        return view("welcome");
    }
}
```

## Appel depuis la route:

Maintenant nous devons faire le lien entre notre fichier **web.php** et notre Controller.

Nous devons donc changer notre deuxième paramètre :

```
Route::get('/', [UserController::class, 'index']);
```

•A partir de Laravel 8.x, l'appel d'une action d'un contrôleur s'effectue en passant un tableau comme deuxième paramètre de la fonction; Le tableau comprend le nom de la classe contrôleur et le nom de la méthode à appeler.

## Appel depuis la route: Utilisation des paramètres

Ajoutons à notre contrôleur la méthode **get** prenant en argument un parameter $id:

```
class UserController extends Controller
{
    public function get($id) {
        return "page $id";
    }
}
```

Ajoutons dans le fichier ***web.php*** *la route suivante*:

```
Route::get('/get/{id}', [UserController::class, 'get']);
```

## Contrôleurs à simple action (invokable)

- Un **controller invokable** est un contrôleur qui est conçu pour gérer une seule action. Cela est possible grâce à la méthode magique **__invoke(),** qui permet de rendre le contrôleur appelable comme une fonction. Cela signifie qu'elle est utilisée pour une action spécifique (par exemple, afficher une page, traiter un formulaire, etc.) au lieu de regrouper plusieurs actions comme dans un contrôleur classique.

- Vous pouvez générer un contrôleur invocable en utilisant l' option **--invokable** de la commande Artisan **make:controlle**r :

**php artisan make:controller NomDuController --invokable**

**Exemple :** **php artisan make:controller WelcomeController --invokable**

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class WelcomeController extends Controller
{
    public function __invoke()
    {
        return view('welcome');
    }
}
```

Dans cet exemple :Lorsque ce contrôleur est appelé, il retourne la vue welcome.

```php
use App\Http\Controllers\WelcomeController;

Route::get('/', WelcomeController::class);
```

Au lieu de spécifier une méthode comme **WelcomeController@index**, vous passez directement le nom de la classe. Laravel comprendra que cette classe contient une méthode **__invoke()**.

## Exemple 2 : Contrôleur invokable avec logique métier

Un contrôleur qui retourne un message personnalisé :

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class GreetingController extends Controller
{
    public function __invoke(Request $request)
    {
        $name = $request->input('name', 'Invité');
        return "Bonjour, $name!";
    }
}
```

Dans les routes :

```php
use App\Http\Controllers\GreetingController;

Route::get('/greet', GreetingController::class);
```

Quand vous accédez à **/greet?name=Ahmed**, cela affiche : **Bonjour, Ahmed!**

## Contrôleur de ressources:

Un **contrôleur resource** en Laravel est une classe contrôleur qui est configurée pour gérer les opérations **CRUD (Create, Read, Update, Delete)** sur une ressource donnée.

Un contrôleur resource contient des méthodes préconfigurées pour effectuer les opérations suivantes sur une ressource :

1. **index :** Afficher une liste des ressources.
2. **create :** Afficher un formulaire pour créer une nouvelle ressource.
3. **store :** Enregistrer une nouvelle ressource dans la base de données.
4. **show :** Afficher une ressource spécifique.
5. **edit :** Afficher un formulaire pour éditer une ressource existante.
6. **update :** Mettre à jour une ressource dans la base de données.
7. **destroy :** Supprimer une ressource.

Vous pouvez créer un contrôleur resource à l'aide de la commande **Artisan** suivante :

**php artisan make:controller NomDuController --resource**

**Exemple :** **php artisan make:controller ProductController --resource**

Cette commande générera un contrôleur sous **app/Http/Controllers/ProductController.php**. Le contrôleur contiendra une méthode pour chacune des opérations de ressources disponibles.

## Contrôleur de ressources

La classe **ProductController** créée a l'allure suivante:

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProductController extends Controller
{
    // 1. Affiche une liste des produits
    public function index()
    {
        return "Liste des produits";
    }

    // 2. Affiche le formulaire pour créer un produit
    public function create()
    {
        return "Formulaire de création de produit";
    }

    // 3. Enregistre un nouveau produit dans la base de données
    public function store(Request $request)
    {
        return "Enregistrer un produit";
    }

    // 4. Affiche un produit spécifique
    public function show($id)
    {
        return "Afficher le produit avec l'ID : $id";
    }

    // 5. Affiche le formulaire pour éditer un produit existant
    public function edit($id)
    {
        return "Formulaire d'édition pour le produit avec l'ID : $id";
    }

    // 6. Met à jour un produit dans la base de données
    public function update(Request $request, $id)
    {
        return "Mettre à jour le produit avec l'ID : $id";
    }

    // 7. Supprime un produit de la base de données
    public function destroy($id)
    {
        return "Supprimer le produit avec l'ID : $id";
    }
}
```

## Contrôleur de ressources

La méthode **resource** () de la classe **Route** est une fonction statique comme la méthode **get** () qui donne accès aux différentes routes que nous pouvons utiliser dans un contrôleur.

Syntaxe de la méthode resource() :

```
Route::resource("/products", ProductController::class);
```

Cette déclaration de route unique crée plusieurs routes pour gérer diverses actions sur la ressource.

Le contrôleur généré contient déjà les méthodes souhaitées pour chacune de ces actions.

N'oubliez pas que vous pouvez toujours obtenir un aperçu rapide des routes de votre application en exécutant la commande Artisan **route:list**.

En excécutant la commande ci-après, on obtient:

```
php artisan route:list --except-vendor
```

```
GET|HEAD      products ..... products.index › productController@index
product       products ..... products.store › productController@store
GET|HEAD      products/create ..... products.create › productController@create
GET|HEAD      products/{product} ..... products.show › productController@show
PUT|PATCH     products/{product} ..... products.update › productController@update
DELETE        products/{product} ..... products.destroy › productController@destroy
GET|HEAD      products/{product}/edit ..... products.edit › productController@edit
```

## Contrôleur de ressources

Vous pouvez même enregistrer plusieurs contrôleurs de ressources à la fois en passant un tableau à la méthode **resources** :

```
Route::resources([
'photos' => PhotoController::class,
'products' => ProductController::class,
]);
```